RISING WATERS

Abstract

Floods are one of the most devastating natural disasters, causing significant damage to human lives, property, agriculture, transportation, and the environment. Timely prediction of flood conditions plays an important role in reducing these losses by enabling early warning and effective disaster management. With the advancement of machine learning, it has become possible to analyze environmental data and predict flood conditions with improved accuracy and speed. This project, "Rising Water Prediction System Using Machine Learning," is developed to demonstrate how machine learning techniques can assist in predicting flood alerts based on environmental factors.

The proposed system is implemented using Python and provides an interactive web application through the Streamlit framework. The application accepts four important environmental parameters as input: Rainfall, Water Level, Temperature, and Humidity. These parameters are commonly associated with flood conditions and are used by the machine learning model to determine whether a flood alert should be generated. Users can enter the values through a simple interface, and the system instantly displays the prediction as "Flood Alert: YES" or "Flood Alert: NO."

The machine learning model used in this project is the Decision Tree Classifier, which is trained on a sample flood dataset. The dataset is preprocessed using the Pandas library, while model training, testing, and evaluation are performed using Scikit-learn. The model's performance is evaluated using an accuracy score and a classification report, helping users understand its effectiveness on the available dataset. The application also includes a graphical visualization of Rainfall versus Water Level using Matplotlib, allowing users to observe the relationship between these environmental factors.

The main objective of this project is to provide a simple, educational, and user-friendly flood prediction system that demonstrates the practical application of machine learning in disaster prediction. Although the current implementation is based on a sample dataset, the project establishes a strong foundation for future improvements such as integrating real-time weather APIs, larger datasets, cloud deployment, mobile applications, and automated alert systems.

Problem Statement

Floods cause significant loss of life, property damage, and economic disruption every year. Disaster management authorities need an accurate and timely flood prediction system to issue early warnings and plan effective emergency responses. This customer problem statement helps identify the challenges faced by these authorities and focuses on developing a machine learning-based solution that predicts flood risk using historical weather data. Understanding the customer's needs enables the team to build a reliable, user-friendly system that supports early warning, evacuation planning, and efficient disaster management.

Features of the Rising Water Prediction System

The Rising Water Prediction System is designed to provide an efficient and user-friendly platform for predicting flood conditions using machine learning techniques. The system combines data analysis, predictive modeling, and visualization to help users understand flood risk based on environmental conditions. The key features of the project are described below.

1. User-Friendly Interface

The application provides a simple and interactive interface developed using Streamlit. Users can easily enter environmental parameters without requiring any technical knowledge. The clean layout and organized input fields make the system easy to understand and operate for students, researchers, and general users.

2.Environmental Data Input

The system accepts four important environmental parameters as input:
Rainfall (mm): Indicates the amount of rainfall received in a particular area.

Water Level (m):

Represents the height of the water level in rivers, lakes, or reservoirs.

Temperature (°C):

Helps analyze weather conditions that may influence rainfall patterns.

Humidity (%):

Represents the moisture content in the atmosphere, which affects precipitation.

3. Flood Prediction Using Machine Learning

The core feature of the application is the flood prediction module. The system uses the Decision Tree Classifier algorithm from the Scikit-learn library to analyze the user-provided data. Based on the trained model, the application predicts whether the given environmental conditions indicate a potential flood situation

4. Instant Flood Alert

After processing the input values, the system immediately displays the prediction result as:
Flood Alert: YES – Indicates that the environmental conditions suggest a high possibility of flooding.
Flood Alert: NO – Indicates that the current conditions are less likely to cause flooding.

Tech Stack

The Rising Water Prediction System is developed using Python and several powerful libraries that support machine learning, data analysis, visualization, and web application development.

Python:

Python is the primary programming language used to develop the entire application. It is simple, efficient, and widely used for machine learning and data science projects.

Streamlit:

Streamlit is used to create the web application interface. It allows users to enter rainfall, water level, temperature, and humidity values and instantly receive the flood prediction result.

Pandas:

Pandas is used to create and manage the dataset. It helps organize, process, and prepare the data before training the machine learning model.

Scikit-learn:

Scikit-learn provides the Decision Tree Classifier algorithm used to train the model, make predictions, and evaluate the model's performance using accuracy and a classification report.

Matplotlib:

Matplotlib is used to generate the Rainfall vs Water Level line graph, which helps visualize the relationship between the two variables.

Google Colab:

Google Colab is used as the development environment to write, train, test, and execute the project code online.

LocalTunnel:

LocalTunnel is used to generate a public URL, allowing the Streamlit application to be accessed through a web browser

Installation Steps

The following steps explain how to install and run the Rising Water Prediction System successfully.

Step 1:

Install Python
Download and install Python 3.x from the official Python website. During installation, ensure that the "Add Python to PATH" option is selected so that Python commands can be executed from the command prompt.

Step 2:

Open Google Colab

Open Google Colab using a web browser and create a new notebook. Google Colab provides a cloud-based environment where the project can be developed and executed without installing additional software on the local computer

Step 3:

Install Required Libraries

Install the required Python libraries before running the project. Execute the following commands:

!pip uninstall -y streamlit
!pip install streamlit==1.35.0
!pip install pandas
!pip install matplotlib
!pip install scikit-learn
!pip install localtunnel

These libraries are required for:

Pandas – Data handling
Matplotlib – Graph generation
Scikit-learn – Machine learning model
Streamlit – Web application
LocalTunnel – Public URL generation

Step 4:

Write the Python Code

Write the machine learning code to:

Import the required libraries.
Create the flood dataset.
Train the Decision Tree Classifier.
Test the model.
Display the accuracy and classification report.
Generate the Rainfall vs Water Level graph.

Step 5:

Create the Streamlit Application

Create a file named app.py and write the Streamlit code. The application should include input fields for:
Rainfall (mm)
Water Level (m)
Temperature (°C)
Humidity (%)
It should also contain a Predict button to display the flood prediction result.

Step 6:

Run the Streamlit Application
Execute the following command to start the Streamlit server:

```
!streamlit run app.py &>/content/logs.txt &
```

This command launches the web application in the background.

Step 7:

Generate a Public URL
Since Streamlit runs locally in Google Colab, generate a public URL using LocalTunnel:

```
!npx localtunnel --port 8501
```

A public URL will be displayed. Open this link in any web browser to access the Rising Water Prediction System.

Step 8:

Test the Application
Enter sample values for Rainfall, Water Level, Temperature, and Humidity, then click the Predict button. The application displays either:
Flood Alert: YES, or Flood Alert: NO

Usage Instructions

The following steps explain how to use the Rising Water Prediction System after the application has been installed and launched.

Step 1:

Launch the Application

Run the Streamlit application using the command provided in the installation steps. After generating the LocalTunnel URL, open it in a web browser. The homepage of the Rising Water Prediction System will be displayed with input fields for environmental parameters.

Step 2:

Enter Environmental Parameters
The application provides four input fields. Enter the required values in each field:

Rainfall (mm):

Enter the amount of rainfall in millimeters.

Water Level (m):

Enter the current water level in meters.

Temperature (°C):

Enter the current temperature.

Humidity (%):

Enter the atmospheric humidity percentage.
Ensure that all values are entered correctly before proceeding.

Step 3:

Click the Predict Button
After entering all the required values, click the Predict button. The application processes the entered data and performs flood prediction.

Step 4:

View the Prediction Result
Based on the entered environmental conditions, the application displays one of the following results:
Flood Alert: YES – Indicates a high possibility of flooding.
Flood Alert: NO – Indicates that the entered conditions do not suggest a flood.

Step 5:

Analyze the Model Performance
The project also displays the Accuracy Score and Classification Report, which help evaluate the performance of the Decision Tree model. These metrics indicate how effectively the model predicts flood conditions using the available dataset.

Step 6:

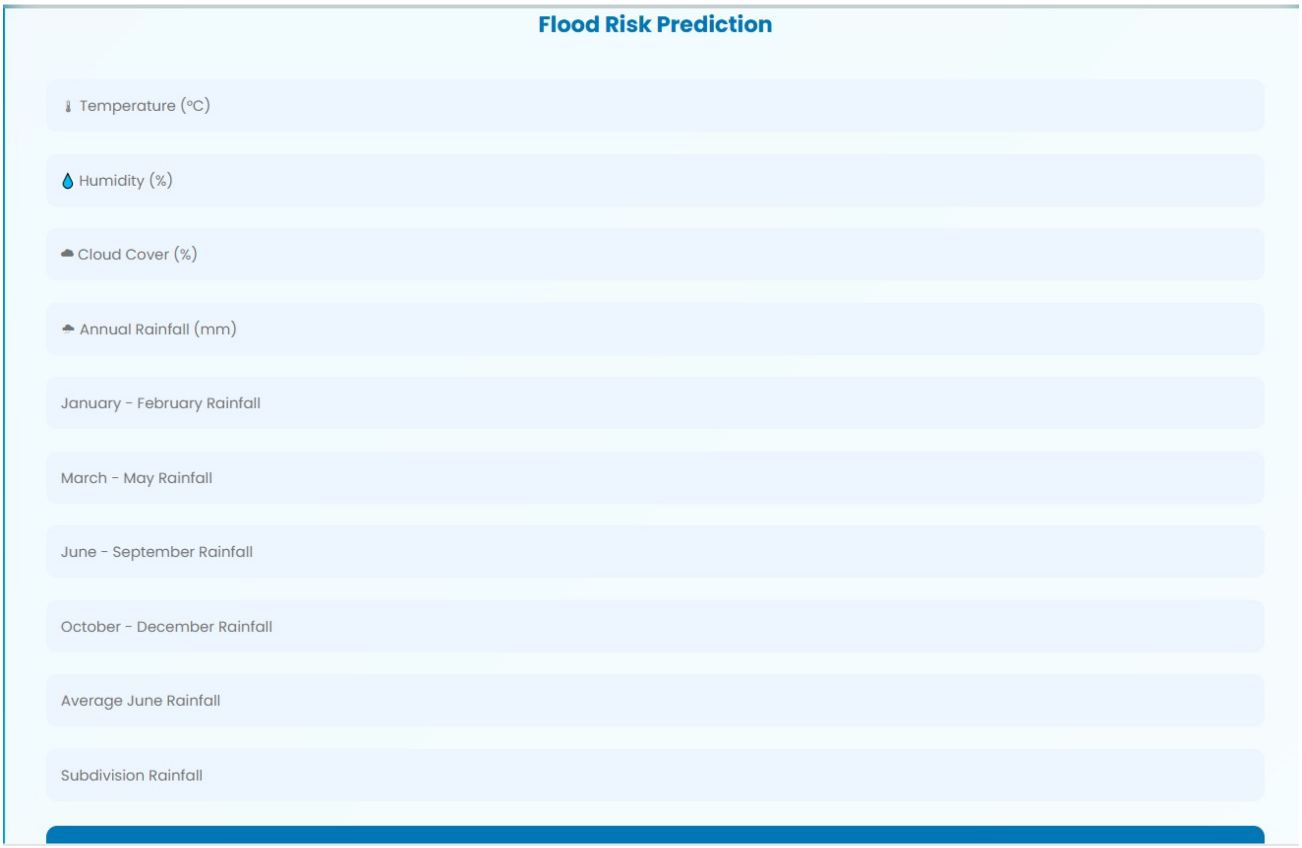
View the Graph
The application generates a Rainfall vs Water Level line graph. This graph visually represents the relationship between rainfall and water level, making it easier to understand the data used in the prediction.

Step 7:

Repeat the Prediction
Users can enter different values for rainfall, water level, temperature, and humidity to test various scenarios. By clicking the Predict button again, the application generates a new prediction without restarting the program.

Expected Output

After successful execution, the system:
Accepts environmental data from the user.
Predicts flood conditions using the trained Decision Tree model.
Displays Flood Alert: YES or Flood Alert: NO.
Shows the model accuracy and classification report.
Displays the Rainfall vs Water Level graph for visualization.


Flood Risk Prediction
🌡 Temperature (°C)
💧 Humidity (%)
☁ Cloud Cover (%)
🌧 Annual Rainfall (mm)
January - February Rainfall
March - May Rainfall
June - September Rainfall
October - December Rainfall
Average June Rainfall
Subdivision Rainfall

High Flood Risk

⚠ High Chance of Flood

Recommended Actions

- Move to higher ground if required.
- Follow official weather alerts.
- Keep emergency supplies ready.
- Avoid driving through flooded roads.
- Protect important documents.

🔄 Predict Again

Future Scope

The Rising Water Prediction System provides a basic machine learning solution for predicting flood conditions using environmental parameters. Although the current project successfully demonstrates the use of a Decision Tree Classifier for flood prediction, there are several opportunities to improve and extend the system in the future.

1. Integration with Real-Time Weather Data

The current system uses a sample dataset for prediction. In the future, it can be integrated with real-time weather APIs such as OpenWeatherMap or government meteorological services. This will enable the application to collect live rainfall, temperature, humidity, and water level data, providing more accurate and timely flood predictions.

2. Use of Larger and More Diverse Datasets

The prediction accuracy can be improved by training the model on larger datasets containing historical flood records from different regions. A diverse dataset will help the model recognize more complex flood patterns and improve its performance under different environmental conditions.

3. Advanced Machine Learning Models

The current project uses the Decision Tree Classifier. In the future, advanced machine learning algorithms such as Random Forest, XGBoost, or Gradient Boosting can be implemented and compared to achieve higher prediction accuracy and better reliability.

4. Mobile Application Development

The system can be extended into an Android or iOS mobile application. A mobile app would allow users to access flood predictions from anywhere, making the system more convenient and useful during emergency situations.

5. Cloud Deployment

Currently, the application is executed locally using Streamlit and LocalTunnel. In the future, it can be deployed on cloud platforms such as

Streamlit Community Cloud, Render, or Google Cloud. This will make the application accessible to users through a permanent online URL

6. Automatic Alert and Notification System

An automated alert system can be integrated to send notifications through SMS, email, or mobile push notifications whenever the predicted flood risk is high. This feature can help authorities and the public take timely preventive actions.

7. Enhanced Data Visualization

Future versions of the application can include interactive dashboards, multiple charts, and trend analysis to help users better understand flood-related environmental data and prediction result

8. Integration with GIS and Maps

The project can be enhanced by integrating Geographic Information Systems (GIS) and interactive maps to display flood-prone locations visually. This will help users identify areas at higher risk and improve disaster management planning.

9. Multi-Region Flood Prediction

The application can be expanded to support predictions for multiple cities, districts, or states by incorporating regional datasets. This will make the system suitable for broader real-world applications.

10. Improved User Experience

Future improvements may include a more attractive user interface, multilingual support, better input validation, and additional features that make the application easier to use for a wider range of users.

Submitted by:-

Putta Jhashwanth-Team Member

Vellampalli Jyothsna-Team Member

Sakam Harshitha-Team Member